

DATA ENGINEERING COURSE

Introduction to Pandas

Class Notes & Reference Material

Instructor: Tanishq Tyagi
Data Engineering Course

Table of Contents

1. Session Overview & Learning Objectives
2. Why Do We Need Pandas?
 - 2.1 The Problem with Raw Python Lists
 - 2.2 NumPy vs Pandas
 - 2.3 Why Data Engineers Love Pandas
3. Series vs DataFrame
 - 3.1 What is a Series?
 - 3.2 Custom Indexes in Series
 - 3.3 What is a DataFrame?
 - 3.4 Series vs DataFrame Comparison
4. Reading Data into Pandas
 - 4.1 Reading CSV Files
 - 4.2 Reading Excel Files
 - 4.3 Reading JSON Files
 - 4.4 Best Practices for Reading Data
5. Understanding Dataset Structure
 - 5.1 head() and tail()
 - 5.2 dtypes
 - 5.3 info()
 - 5.4 describe()
 - 5.5 Custom Aggregations
6. Practical Examples
7. Real-World Data Engineering Perspective
8. Summary & Quick Revision Sheet
9. General Interview Questions — Beginner & Intermediate
10. Data Engineer Interview Preparation

SECTION 1 — SESSION OVERVIEW & LEARNING OBJECTIVES

1. Session Overview

In this session we explored **pandas** — Python's most powerful library for working with structured data. We started by understanding *why* pandas exists and what problems it solves that plain Python lists and even NumPy cannot handle conveniently. We then covered the two core data structures — **Series** and **DataFrame** — and practiced reading real-world data from CSV, Excel, and JSON files. Finally, we used pandas' built-in methods to inspect, summarise, and understand any dataset quickly.

Learning Objectives

- ✓ Explain why pandas is essential for data engineering work and how it improves on plain Python lists and NumPy.
- ✓ Create and manipulate pandas Series objects, including using custom indexes.
- ✓ Create and work with DataFrames as collections of labeled Series.
- ✓ Read data from CSV, Excel, and JSON files into a pandas DataFrame.
- ✓ Use `head()`, `tail()`, `dtypes`, `info()`, and `describe()` to explore any dataset.
- ✓ Apply custom aggregation functions using `agg()`.
- ✓ Connect pandas usage to real-world data engineering pipelines and ETL workflows.

SECTION 2 — WHY DO WE NEED PANDAS?

2. Why Do We Need Pandas?

2.1 The Problem with Raw Python Lists

Imagine Flipkart processes 50 million orders a day. Each order has an ID, a price, a city, a product, a customer, and dozens of other attributes. Storing all of this in separate Python lists creates an immediate problem:

```
# 3 separate lists – already hard to manage
order_id = [1001,      1002,      1005]
prices    = [500,      600,      700]
cities    = ["Delhi", "Mumbai", "Pune"]

# What if you want to filter orders from Delhi with price > 550?
# You must manually loop and cross-reference 3 lists – error-prone and slow.
```

■ The Core Problem

With multiple Python lists: (1) No shared index — lists can fall out of sync. (2) No column labels — you must remember which list means

Pandas solves all of these problems by giving you a structure that looks and behaves like a spreadsheet (Excel) inside Python — but with powerful analytical superpowers.

2.2 NumPy vs Pandas

Before pandas, data engineers used NumPy for numerical work. NumPy is excellent for mathematical operations on arrays and matrices — but it has significant limitations for real-world data:

Feature	NumPy	Pandas
Data types in one structure	Single type only	Mixed types (int, str, float, date)
Column labels	No labels (index only)	Named columns
Missing value handling	Limited	First-class support (NaN)
Reading CSV/Excel/JSON	Not built-in	Built-in (read_csv, read_excel, read_json)
Data cleaning	Manual and verbose	Rich built-in methods
Aggregation / GroupBy	Manual loops	groupby(), agg(), pivot_table()
Best used for	Math, matrices, ML arrays	Data wrangling, ETL, analysis

2.3 Why Data Engineers Love Pandas

In a typical data pipeline, data arrives in raw form from many sources. Pandas is the go-to tool for the **Extract** → **Clean** → **Transform** → **Load** stages:

Stage	Tool / Action	Example
Extract	pd.read_csv / read_excel / read_json	Load raw sales data from a CSV
Clean	dropna(), fillna(), astype()	Remove nulls, fix data types

Transform	apply(), groupby(), merge()	Calculate revenue by region
Metrics	describe(), agg(), pivot_table()	Generate summary statistics
Load / Store	to_csv(), to_parquet(), SQLAlchemy	Write to warehouse or database

SECTION 3 — SERIES vs DATAFRAME

3. Series vs DataFrame

3.1 What is a Series?

A **Series** is a one-dimensional labeled array. Think of it as a single column of an Excel spreadsheet. Every value in a Series has an **index label** attached to it.

```
import pandas as pd

# Default integer index (0, 1, 2 ...)
students = pd.Series(["Vibhore", "Aditi", "Manav"])
print(students)

# Output:
# 0    Vibhore
# 1      Aditi
# 2      Manav
# dtype: object
```

3.2 Custom Indexes in Series

One of the most powerful features of a Series is the ability to assign **meaningful labels** as the index instead of plain integers:

```
# Custom index – each student gets an employee ID
students = pd.Series(
    ["Vibhore", "Aditi", "Manav"],
    index=["EMP101", "EMP102", "EMP103"]
)
print(students)

# Output:
# EMP101    Vibhore
# EMP102      Aditi
# EMP103      Manav
# dtype: object

# Access by label
print(students["EMP103"]) # Output: Manav
```

■ Key Insight

Custom indexes make data retrieval intuitive. Instead of remembering that index 2 is 'Manav', you can access it by the meaningful label.

3.3 What is a DataFrame?

A **DataFrame** is a two-dimensional labeled table — like a full Excel spreadsheet or a database table. It is made up of multiple Series objects sharing the same index, where each Series represents one column.

```
# A DataFrame is a dictionary of Series
df = pd.DataFrame({
    "ID":    [1, 2],          # Column 1 – a Series
    "Name":  ["Sachi", "Ayush"], # Column 2 – a Series
    "Age":   [21, 22]         # Column 3 – a Series
})
print(df)

# Output:
#    ID  Name  Age
# 0   1  Sachi   21
# 1   2  Ayush   22
```

3.4 Series vs DataFrame — Side-by-Side Comparison

Property	Series	DataFrame
Dimensions	1D (single column)	2D (rows and columns)
Analogy	One column of a spreadsheet	Full spreadsheet / database table
Index	Row index only	Row index + column labels
Created with	pd.Series([...])	pd.DataFrame({...})
Contains	Single data type	Multiple data types across columns
Access element	s[index_label]	df['column'][index_label]
Relationship	—	A DataFrame is a collection of Series

SECTION 4 — READING DATA INTO PANDAS

4. Reading Data into Pandas

One of pandas' greatest strengths is its ability to ingest data from virtually any source with a single line of code. The three most common formats in data engineering are CSV, Excel, and JSON.

4.1 Reading CSV Files

```
import pandas as pd

# Read a CSV file – most common format in data engineering
df = pd.read_csv("data/students.csv")

# Useful parameters:
df = pd.read_csv(
    "data/students.csv",
    sep=",",          # delimiter (default is comma)
    header=0,         # row number for column names
    index_col=None,   # column to use as index
    encoding="utf-8"  # text encoding
)
```

4.2 Reading Excel Files

■ Dependency Required

Reading Excel files requires the openpyxl library. Install it with: `pip install openpyxl`

```
# Read an Excel file
# Requires: pip install openpyxl
df = pd.read_excel("data/contact_responses.xlsx")

# With additional options:
df = pd.read_excel(
    "data/contact_responses.xlsx",
    sheet_name="Sheet1", # specify which sheet to read
    header=0,            # row index for column names
    usecols="A:E"        # only read columns A through E
)
```

4.3 Reading JSON Files

```
# Read a JSON file
df = pd.read_json("data/student.json")

# JSON is common in APIs and NoSQL data sources
# Pandas automatically parses arrays of objects into rows
```

4.4 Best Practices for Reading Data

- Always specify encoding='utf-8' to avoid character encoding errors on non-ASCII data.
- Use usecols to load only the columns you need — this significantly reduces memory usage on large files.
- Use dtype parameter to pre-define column types and avoid automatic (sometimes wrong) type inference.
- For very large files, use chunksize to read in batches: `pd.read_csv('file.csv', chunksize=10000)`.
- Always check `df.shape` and `df.dtypes` immediately after loading to confirm the data loaded correctly.

SECTION 5 — UNDERSTANDING DATASET STRUCTURE

5. Understanding Dataset Structure

Before doing any analysis or transformation on a dataset, a data engineer must first understand its structure. Pandas provides several built-in methods to inspect a DataFrame at a glance.

5.1 head() and tail()

These methods return the first or last N rows of a DataFrame. Essential when working with large datasets where printing the entire DataFrame is impractical.

```
# View first 5 rows (default)
print(df.head())

# View first 3 rows
print(df.head(3))

# View last 5 rows
print(df.tail())

# View last 2 rows
print(df.tail(2))
```

5.2 dtypes — Column Data Types

The **dtypes** attribute shows the data type of each column. This is crucial for catching type-related issues early — for example, a numeric column incorrectly stored as object (string).

```
print(df.dtypes)

# Example output:
# student_id      int64
# name            object
# age             int64
# marks          float64
# grade           object
# dtype: object
```

5.3 info() — Full Dataset Overview

The **info()** method provides a comprehensive summary of the DataFrame in one call. It is the first thing a data engineer runs after loading a new dataset.

```
df.info()

# Output includes:
# - Total number of rows (RangeIndex)
# - Number of columns
# - Column names
# - Non-null count per column (identifies missing values)
# - Data type of each column
# - Total memory usage

# Example output:
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 1000 entries, 0 to 999
# Data columns (total 5 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   student_id  1000 non-null    int64
# 1   name         998 non-null     object  <- 2 missing values
# 2   age          1000 non-null    int64
# 3   marks        995 non-null     float64  <- 5 missing values
# 4   grade        1000 non-null    object
# dtypes: float64(1), int64(2), object(2)
# memory usage: 39.2+ KB
```

■ Pro Tip

The Non-Null Count column in `info()` output is your first signal about data quality. Any column where Non-Null Count < total rows has missing data.

5.4 `describe()` — Summary Statistics

The **`describe()`** method generates descriptive statistics for all numeric columns. It answers questions like: What is the range? Is the data normally distributed? Are there outliers?

```
# Default – numeric columns only
print(df[["Age", "Marks"]].describe())

# Output:
#           Age      Marks
# count  1000.000  995.000  <- number of non-null values
# mean     21.540   72.340  <- average
# std       2.100   15.200  <- standard deviation
# min      18.000   10.000  <- minimum value
# 25%      20.000   62.000  <- 25th percentile (Q1)
# 50%      21.000   74.000  <- median (Q2)
# 75%      23.000   84.000  <- 75th percentile (Q3)
# max      30.000  100.000  <- maximum value
```

Statistic	Meaning	Why It Matters
count	Number of non-null values	Reveals missing data immediately

mean	Average value	Central tendency — can be skewed by outliers
std	Standard deviation	Spread of data around the mean
min / max	Smallest / largest value	Identify outliers and data range
25% (Q1)	25th percentile	25% of values fall below this
50% (Q2)	Median	Middle value — robust to outliers unlike mean
75% (Q3)	75th percentile	75% of values fall below this

Custom Percentiles with describe()

```
# Specify custom percentiles
df.describe(percentiles=[0.10, 0.50, 0.80])

# This shows the 10th, 50th, and 80th percentile
# instead of the default 25th, 50th, 75th
# Useful for SLA analysis (e.g., p90, p95, p99 response times)
```

5.5 Custom Aggregations with agg()

The **agg()** method allows you to apply multiple aggregation functions at once — including your own custom functions.

```
# Define a custom aggregation function
def data_range(column):
    return column.max() - column.min()

# Apply built-in and custom aggregations together
df.agg(["mean", "std", data_range])

# Output:
#           Age      Marks
# mean      21.54      72.34
# std        2.10      15.20
# data_range 12.00      90.00
```

SECTION 6 — PRACTICAL EXAMPLES

6. Practical Examples

Example 1 — End-to-End Dataset Exploration

```
import pandas as pd

# Step 1: Load the dataset
df = pd.read_csv("data/students.csv")

# Step 2: Quick peek at the data
print(df.head(3))
# Shows first 3 rows so we can see column names and sample values

# Step 3: Check data types and missing values
df.info()
# Reveals: total rows, column types, non-null counts, memory

# Step 4: Statistical summary
print(df.describe())
# Shows count, mean, std, min, max, quartiles for numeric columns

# Step 5: Check individual column types
print(df.dtypes)
```

Example 2 — Series with Custom Index

```
import pandas as pd

# Create a Series of city populations with city names as index
city_populations = pd.Series(
    [32_000_000, 21_000_000, 13_000_000, 8_500_000, 7_100_000],
    index=["Delhi", "Mumbai", "Kolkata", "Bangalore", "Chennai"],
    name="Population"
)

print(city_populations)
# Delhi      32000000
# Mumbai     21000000
# Kolkata     13000000
# Bangalore   8500000
# Chennai     7100000
# Name: Population, dtype: int64

# Access by label
print(city_populations["Mumbai"]) # 21000000

# Arithmetic on a Series
print(city_populations / 1_000_000) # Population in millions
```

Example 3 — Building a DataFrame from Scratch

```
import pandas as pd

# Each column is a Series – pandas aligns them by index
df = pd.DataFrame({
    "employee_id": [101, 102, 103, 104],
    "name":         ["Arjun", "Priya", "Rohan", "Sneha"],
    "department":   ["Engineering", "Marketing", "Engineering", "HR"],
    "salary":       [85000, 72000, 91000, 68000],
    "active":       [True, True, False, True]
})

print(df)
print()
print('Shape:', df.shape) # (4, 5)
print('Columns:', df.columns.tolist())
```

SECTION 7 — REAL-WORLD DATA ENGINEERING PERSPECTIVE

7. Real-World Data Engineering Perspective

Where Pandas Appears in Production Pipelines

ETL / ELT Pipelines

Pandas is used in the Extract and Transform phases of ETL. Data engineers read raw files (CSV dumps from databases, API exports, partner data drops), clean them using pandas, and then load the transformed data into a data warehouse like BigQuery, Redshift, or Snowflake.

Data Quality Checks

`df.info()`, `df.describe()`, and `df.isnull().sum()` are used to run automated data quality checks at the start of every pipeline run. These checks catch schema changes, missing data, and type mismatches before they corrupt downstream analytics.

Prototyping Before Spark

For datasets under ~500MB, data engineers prototype transformations in pandas first. Once validated, the same logic is ported to PySpark for production-scale processing on billions of rows.

Microservice Data Processing

Pandas is used inside FastAPI or Flask microservices to process incoming JSON payloads, transform them, and return structured results — for example, a scoring API that reads a customer feature vector and returns a risk score.

Airflow DAG Tasks

Individual tasks in Apache Airflow DAGs often use pandas to process intermediate data between pipeline steps — reading from S3, transforming, and writing back to S3 or a database.

Scalability Considerations

■ Production Warning

Pandas loads the entire dataset into RAM. For datasets larger than ~500MB to 1GB, consider: (1) Chunked reading with `chunksize` parameter.

SECTION 8 — SUMMARY & QUICK REVISION SHEET

8. Summary & Quick Revision Sheet

Key Concepts

- ✓ Pandas is Python's primary data manipulation library — Excel inside Python with analytical superpowers.
- ✓ A Series is a 1D labeled array (like one column). A DataFrame is a 2D table (like an Excel sheet).
- ✓ Every element in a Series has a label (index). Custom indexes make data access intuitive.
- ✓ DataFrames are collections of Series objects sharing the same row index.
- ✓ `pd.read_csv()`, `pd.read_excel()`, `pd.read_json()` are the primary data ingestion functions.
- ✓ `head(n)` / `tail(n)`: preview first/last `n` rows of a large dataset.
- ✓ `dtypes`: check the data type of every column.
- ✓ `info()`: complete overview — rows, columns, types, non-null counts, memory usage.
- ✓ `describe()`: statistical summary — count, mean, std, min, max, quartiles.
- ✓ `agg()`: apply multiple aggregation functions (including custom) simultaneously.

Syntax Cheat Sheet

Action	Code
Import pandas	<code>import pandas as pd</code>
Create Series	<code>pd.Series(..., index=...)</code>
Create DataFrame	<code>pd.DataFrame({'col': [...], ...})</code>
Read CSV	<code>pd.read_csv('file.csv')</code>
Read Excel	<code>pd.read_excel('file.xlsx')</code>
Read JSON	<code>pd.read_json('file.json')</code>
First N rows	<code>df.head(N)</code>
Last N rows	<code>df.tail(N)</code>
Column types	<code>df.dtypes</code>
Full overview	<code>df.info()</code>
Stats summary	<code>df.describe()</code>
Custom stats	<code>df.describe(percentiles=[0.10, 0.90])</code>
Multi-agg	<code>df.agg(['mean', 'std', custom_func])</code>
Shape	<code>df.shape</code> # (rows, cols)
Column names	<code>df.columns.tolist()</code>

SECTION 9 — GENERAL INTERVIEW QUESTIONS

9. General Interview Questions

Beginner Level

Q: What is pandas and why is it used?

A: Pandas is an open-source Python library for data manipulation and analysis. It provides two core data structures — Series and DataFrame — that allow you to store, clean, filter, transform, and analyse structured data efficiently. It is used because it simplifies complex data operations that would otherwise require many lines of manual Python code.

Q: What is the difference between a Series and a DataFrame?

A: A Series is a one-dimensional labeled array (like a single column of data). A DataFrame is a two-dimensional table made up of multiple Series sharing the same row index. Think of a Series as one column of an Excel sheet, and a DataFrame as the full Excel sheet.

Q: What does the index of a Series represent?

A: The index provides a label for each element in a Series. By default, pandas assigns integer labels (0, 1, 2 ...). You can replace these with meaningful labels (strings, dates, IDs) using the index parameter when creating a Series. This makes data lookup intuitive.

Q: How do you create a DataFrame in pandas?

A: The most common way is to pass a Python dictionary to `pd.DataFrame()` where each key is a column name and each value is a list (or array) of data. Example: `pd.DataFrame({'Name': ['A', 'B'], 'Age': [21, 22]})`.

Q: What is the difference between `pd.read_csv()` and `pd.read_excel()`?

A: `pd.read_csv()` reads comma-separated value files (.csv) — no extra dependency needed. `pd.read_excel()` reads Excel files (.xlsx, .xls) and requires the `openpyxl` (or `xlrd` for .xls) library to be installed separately.

Q: What does `df.head()` do?

A: `df.head(n)` returns the first `n` rows of the DataFrame. The default is 5 rows if `n` is not specified. It is used to quickly preview the structure and content of a dataset without printing all rows.

Q: What information does `df.info()` provide?

A: `df.info()` prints: the class type (DataFrame), the RangeIndex (number of rows), the total number of columns, each column's name, non-null count (which reveals missing values), and data type. It also shows total memory usage.

Q: What does `df.describe()` return?

A: `df.describe()` returns summary statistics for all numeric columns: count (non-null values), mean, std (standard deviation), min, 25th percentile (Q1), 50th percentile (median), 75th percentile (Q3), and max.

Q: What is the `dtypes` attribute?

A: `df.dtypes` returns a Series where each element is the data type of the corresponding column in the DataFrame. Common dtypes include `int64` (integer), `float64` (decimal), `object` (string), `bool` (boolean), and `datetime64` (dates).

Q: Why might a numeric column show dtype 'object' in pandas?

A: This happens when the column contains non-numeric values mixed in with numbers — for example, a comma in numbers ('1,000'), currency symbols ('\$500'), or blank strings. Pandas falls back to object type when it cannot parse all values as numbers. The fix is to clean the column and use `astype(float)` or `pd.to_numeric()` to convert it.

Q: How do you access a single element from a Series by its custom label?

A: Using bracket notation with the label: `series['label_name']`. For example, if `students` is a Series with index `['EMP101', 'EMP102', 'EMP103']`, then `students['EMP102']` returns the value at that label.

Intermediate Level

Q: What is the difference between `df.describe()` on numeric vs object columns?

A: By default, `describe()` only includes numeric columns showing count, mean, std, min, max, and quartiles. If you pass `include='object'` or `include='all'`, it also summarises string columns showing count, unique (number of distinct values), top (most frequent value), and freq (frequency of the most common value).

Q: How do you read only specific columns from a CSV to save memory?

A: Use the `usecols` parameter: `pd.read_csv('file.csv', usecols=['col1', 'col2'])`. This tells pandas to only read and load those columns, which can dramatically reduce memory usage for wide datasets with many columns.

Q: What does the `agg()` function do and how is it different from `describe()`?

A: `agg()` lets you specify exactly which aggregation functions to apply — including custom Python functions — and returns only those results. `describe()` always computes a fixed set of statistics. `agg()` is more flexible: `df.agg(['mean', 'std', my_custom_func])`.

Q: What is the purpose of the non-null count in `df.info()`?

A: The non-null count tells you how many values in each column are not missing (NaN). If the non-null count is less than the total number of rows, that column has missing values that need to be handled before analysis or modelling.

Q: What is the difference between `df.shape`, `df.size`, and `len(df)`?

A: `df.shape` returns a tuple (rows, columns). `df.size` returns total number of elements (rows * columns). `len(df)` returns the number of rows only. All three are useful in different contexts.

Q: Can a Series have duplicate index labels?

A: Yes. Unlike dictionary keys, Series index labels do not need to be unique. However, accessing a duplicate label returns all matching values as a new Series rather than a single scalar value. This can cause unexpected behaviour and should be avoided in practice.

Q: How would you read only the first 100 rows of a large CSV?

A: Use the `nrows` parameter: `pd.read_csv('file.csv', nrows=100)`. This is useful for testing your code on a sample before running it on the full dataset.

Q: What is the difference between the 25th percentile and the median in `describe()` output?

A: The 25th percentile (Q1) means 25% of values fall below this number. The median (50th percentile, Q2) means exactly half the values fall below it. The median is preferred over the mean for skewed distributions because it is not affected by extreme outliers.

Q: Why is pandas considered 'in-memory' and what does that mean for large datasets?

A: Pandas loads the entire dataset into RAM. This means the dataset size is limited by your machine's available memory. For files larger than 1-2 GB, pandas may run out of memory or become very slow. In those cases, chunked reading, Polars, Dask, or PySpark should be used.

Q: How do you check for missing values in a DataFrame?

A: `df.isnull().sum()` returns the count of missing (NaN) values per column. `df.isnull()` returns a boolean DataFrame where True means the value is missing. These are the standard first steps in any data quality check.

SECTION 10 — DATA ENGINEER INTERVIEW PREPARATION

10. Data Engineer Interview Preparation

The following questions focus on how pandas is used in real Data Engineering production environments — ETL pipelines, data quality frameworks, Airflow DAGs, and scalable data architectures. These go beyond syntax and test your practical understanding.

Q: How would you use pandas in an ETL pipeline?

A: In an ETL pipeline, pandas handles the Extract and Transform stages. After reading raw data with `pd.read_csv()` or `pd.read_excel()`, we apply transformations such as type casting with `astype()`, null handling with `fillna()` or `dropna()`, and business logic with `apply()` or `merge()`. The transformed DataFrame is then written to a data warehouse using `to_sql()` (via SQLAlchemy) or `to_parquet()` for cloud storage. In Airflow, each of these steps would be an individual `PythonOperator` task.

Q: How do you handle large CSV files in pandas that don't fit in memory?

A: Use the `chunksize` parameter in `pd.read_csv()`: for chunk in `pd.read_csv('big_file.csv', chunksize=100000)`. This returns an iterator, and you process each chunk independently, then either aggregate results or write each chunk to an output. For even larger datasets, switch to Dask (same pandas API, distributed execution) or PySpark.

Q: What is the role of `df.info()` in automated data quality monitoring?

A: In a production pipeline, `df.info()` output can be parsed programmatically to detect schema changes. By comparing column names, dtypes, and non-null counts against an expected schema definition (stored in a YAML or database), you can raise alerts when new columns appear, columns go missing, or null rates exceed thresholds. This is the foundation of data contract validation.

Q: Why would you prefer Parquet over CSV as an intermediate format in a data pipeline?

A: Parquet is a columnar binary format with built-in compression. Compared to CSV: (1) Much smaller file size (often 5-10x), (2) Faster reads especially for wide tables because you only load needed columns, (3) Schema is embedded in the file so dtypes are preserved exactly — no type inference issues, (4) Natively supported by Spark, BigQuery, Redshift, and Athena. Write with `df.to_parquet('file.parquet')`.

Q: How would you use pandas to validate data quality before loading to a warehouse?

A: A standard data quality check pipeline runs: (1) `df.isnull().sum()` — check missing value rates per column against thresholds; (2) `df.dtypes` — verify schema matches expectations; (3) `df.describe()` — check that numeric ranges are within expected bounds; (4) `df.duplicated().sum()` — check for duplicate records; (5) `df['col'].value_counts()` — check categorical distributions. If any check fails, the pipeline raises an exception and stops before loading bad data into the warehouse.

Q: When would you choose pandas over PySpark for data processing?

A: Use pandas when: dataset fits comfortably in RAM (rule of thumb: under 1-2 GB), you need fast prototyping or exploratory analysis, you are building a microservice that processes small payloads, or the team is not Spark-proficient. Use PySpark when: dataset is in the GB-TB range, you need distributed processing across a cluster, you are working in a cloud data platform (Databricks, EMR, Dataproc), or you need fault tolerance for long-running jobs.

Q: How do you use describe() output to detect outliers in a production dataset?

A: Compare the max value against the 75th percentile (Q3) and the IQR (Q3 - Q1). If $\text{max} \gg Q3 + 1.5 * \text{IQR}$, you have upper outliers. Similarly, if $\text{min} \ll Q1 - 1.5 * \text{IQR}$, you have lower outliers. In production monitoring, you can store the describe() output from a reference (clean) dataset, then compare incoming data's describe() against it using tolerance thresholds to flag anomalies automatically.

Q: What is the difference between pd.read_csv() and pd.read_json() in terms of data handling?

A: pd.read_csv() reads flat tabular data — it expects rows and columns. pd.read_json() can handle both flat JSON arrays and nested JSON structures. For nested JSON (e.g., API responses with nested objects), pd.read_json() may flatten only the first level. For deeply nested structures, pd.json_normalize() is needed. In a real pipeline, API data often requires json_normalize() before it can be used as a flat DataFrame.

Q: How would you use agg() in a data pipeline to compute business metrics?

A: agg() is used after groupby() to compute multiple metrics simultaneously. Example: df.groupby('department').agg({'salary': ['mean', 'max', 'count'], 'bonus': 'sum'}). This computes average salary, max salary, headcount, and total bonus per department in one pass — far more efficient than computing each metric separately. In a pipeline, this output would be written to a reporting table in the warehouse.

Q: Explain the concept of a data contract and how pandas can help enforce it.

A: A data contract is a formal agreement about the schema, types, null rates, and value ranges expected in a dataset. Pandas enforces contracts by: reading the data, running df.info() and df.dtypes() to compare against the expected schema, using df.describe() to verify value ranges, and df.isnull().sum() to check null thresholds. Tools like Great Expectations and Soda Core use pandas under the hood for exactly this purpose.

Q: What would you do if a CSV file has inconsistent date formats across rows?

A: After reading with pd.read_csv(), the date column will be dtype object (string). Use pd.to_datetime(df['date'], infer_datetime_format=True) with errors='coerce' to parse what it can and set unparseable values to NaT (Not a Time). Then df['date'].isnull() reveals rows with bad dates. These can be logged, corrected, or dropped depending on business rules.

Q: How does pandas handle missing values when computing describe() statistics?

A: describe() automatically excludes NaN values from all calculations. The count row reflects the number of non-null values, not total rows. This is why count might be lower than the number of rows in the DataFrame — and why you should always check count in the describe() output as your first data quality signal.

Q: What is the difference between df.agg() and df.apply() in a data engineering context?

A: agg() is designed for aggregation — it reduces multiple values to a single value per group (mean, sum, custom function). apply() is more general — it applies a function row-wise or column-wise without necessarily reducing dimensions. In a pipeline, use agg() for summary metrics and apply() for row-level transformations such as cleaning or feature engineering.

Q: How would you read multiple Excel sheets into a single pandas DataFrame?

A: Use pd.read_excel('file.xlsx', sheet_name=None) which returns a dictionary where each key is a sheet name and each value is a DataFrame. Then concatenate them: pd.concat(dfs.values(), ignore_index=True). In a data pipeline, this approach is common when partners submit multi-sheet Excel reports.

Q: If you suspect a column has been silently truncated (values cut off at a fixed length), how would you detect this with pandas?

A: Use df['column'].str.len().describe() to see the distribution of string lengths. If the max length is suspiciously round (e.g., exactly 255 characters) and many values cluster at that length, truncation is likely. This is a real problem with database VARCHAR columns and CSV exports from legacy systems. It is a subtle data quality issue that describe() on the raw column would miss but string length analysis would catch.